FIT2004 - Algorithms and Data Structures

Retrieval Data Structures for Strings

Rafael Dowsley

Introduction

- Suppose you have a large text containing N strings. You want to pre-process it such that searching on this text is efficient.
- Sorting based approach:
  - Pre-processing: sort the string
  - Searching: Binary search to find
- Let M be length of strings. Comparison between two strings: $O(M)$
- Time complexity:
  - Pre-processing: $O(MN \log N)$ using merge sort or $O(MN)$ using radix sort
  - Searching: $O(M \log N)$
- ReTrieval data structures allow answering different queries efficiently

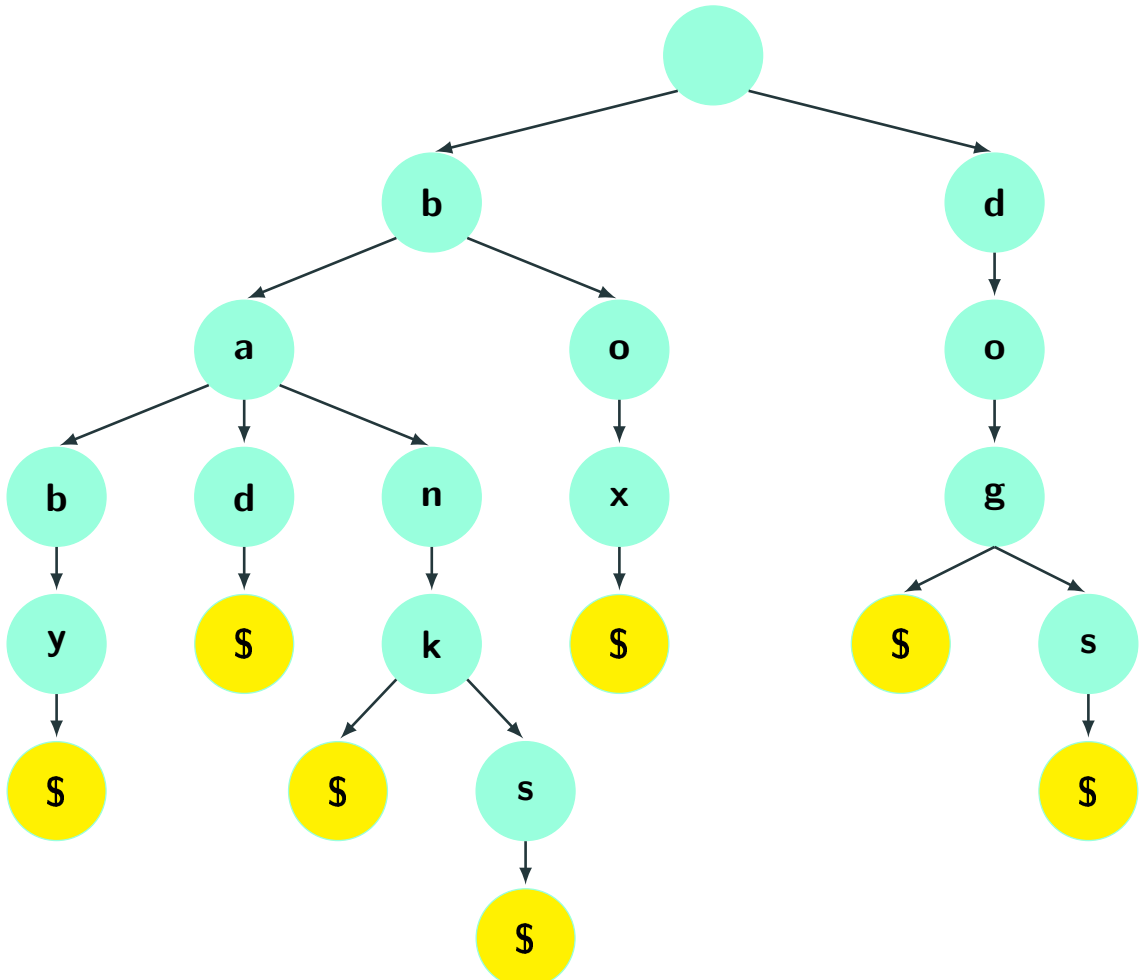
Trie

Trie (ReTRIEval tree)

- Trie is an Σ-way (or multi-way) tree, where Σ is the alphabet size.
  - $\Sigma = 2$ for binary
  - $\Sigma = 26$ for english letters
  - $\Sigma = 4$ for DNA
- Alphabet size could be part of the input to the problem! Unless otherwise specified we are focusing on alphabets of constant size in the slides.
- In a standard trie, all strings with the shared prefix fall within the same subtree/subtrie.
- In fact, it is the shortest possible tree that can be constructed such that all strings with the same prefix fall within the same subtree.

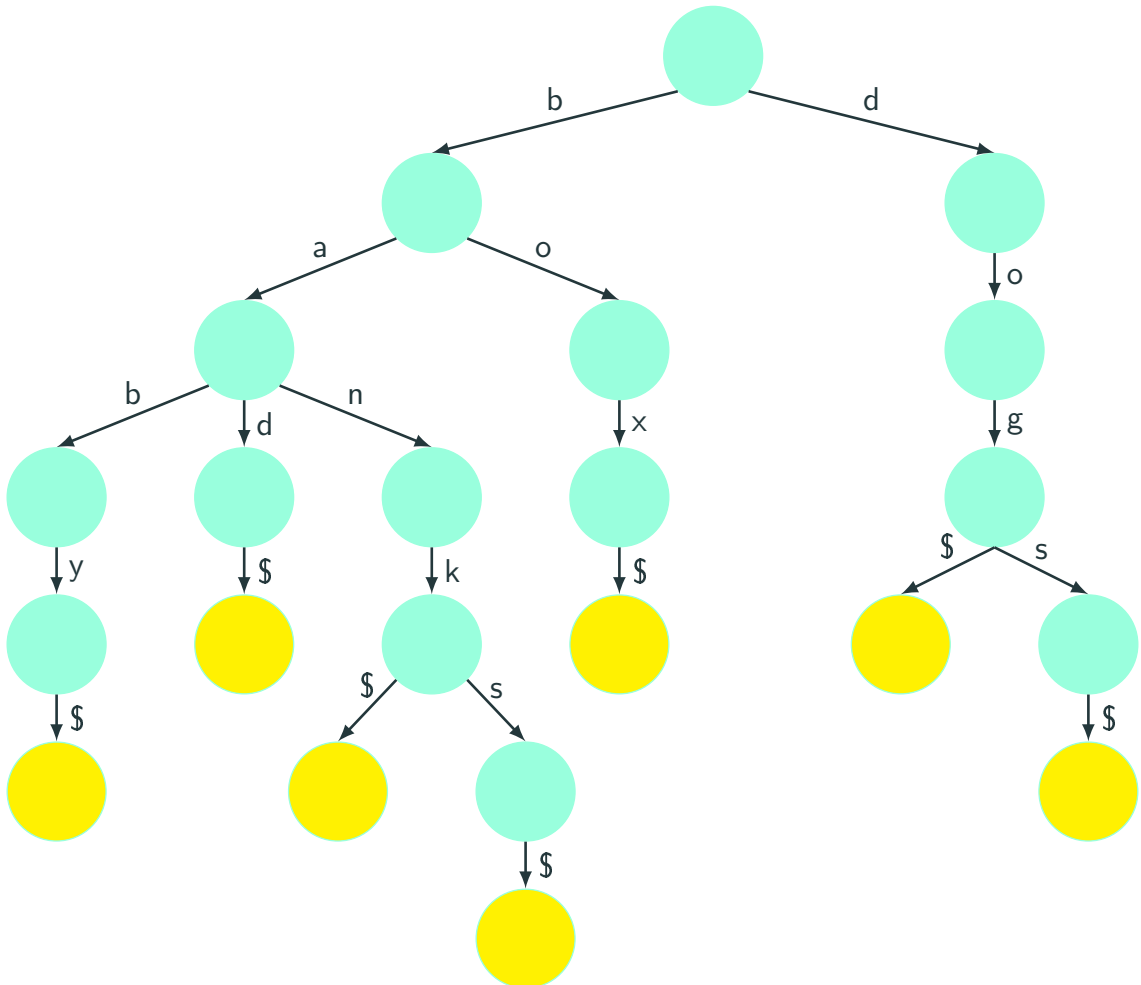
Trie Example

- A trie that stores baby, bad, bank, box, dog, dogs, banks.
- We use $ to denote the end of a string.



Alternative Illustration

- We can also show characters on edges instead of nodes.



Operations on Trie

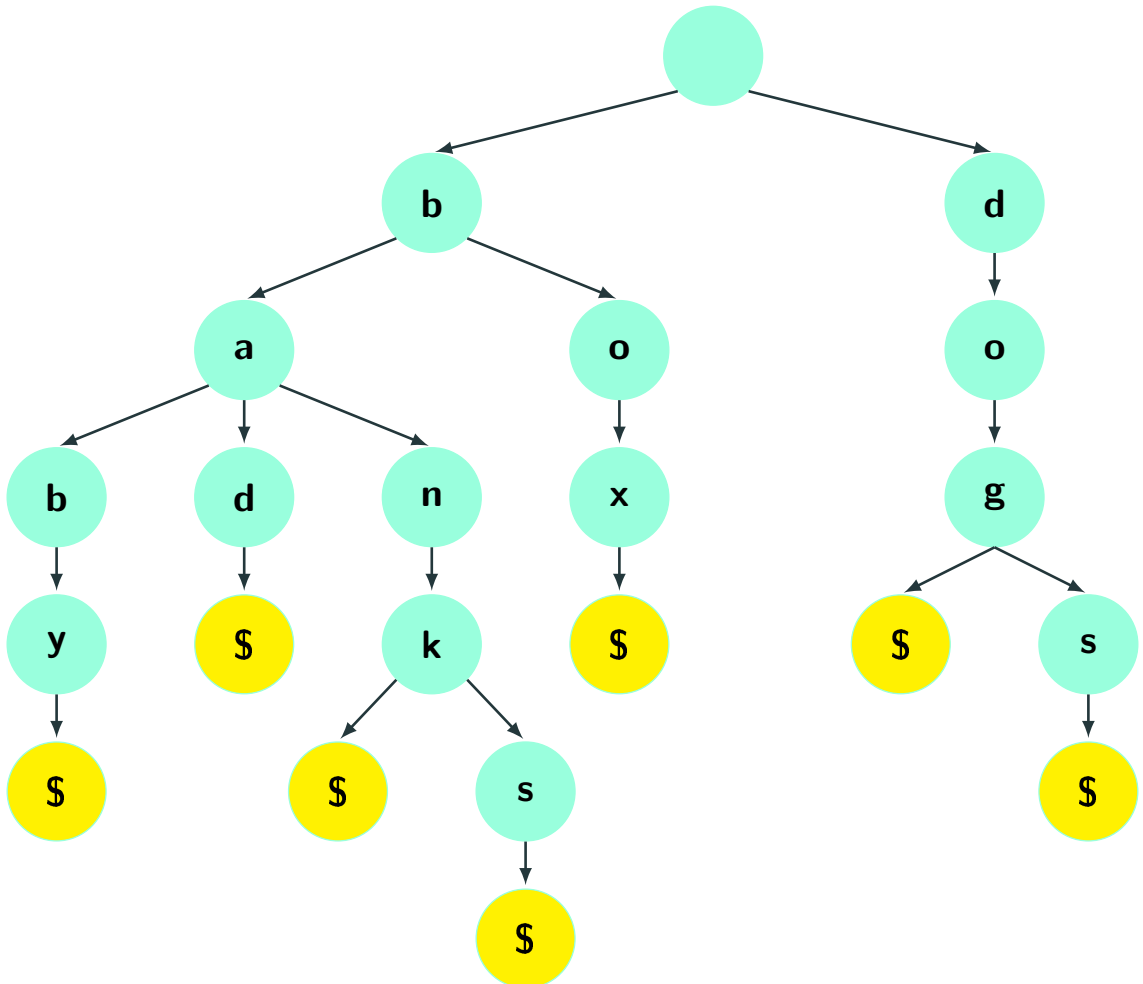
Insert a String

- Start from the root node
- For each character c in the string
  - If a node containing c exists, move to the node.
  - Otherwise, create the node and move to it.

- Time Complexity: $O(M)$ where M is the length of the string.

Operations on Trie

- Insert baby, bad, bank, box, dog, dogs, banks



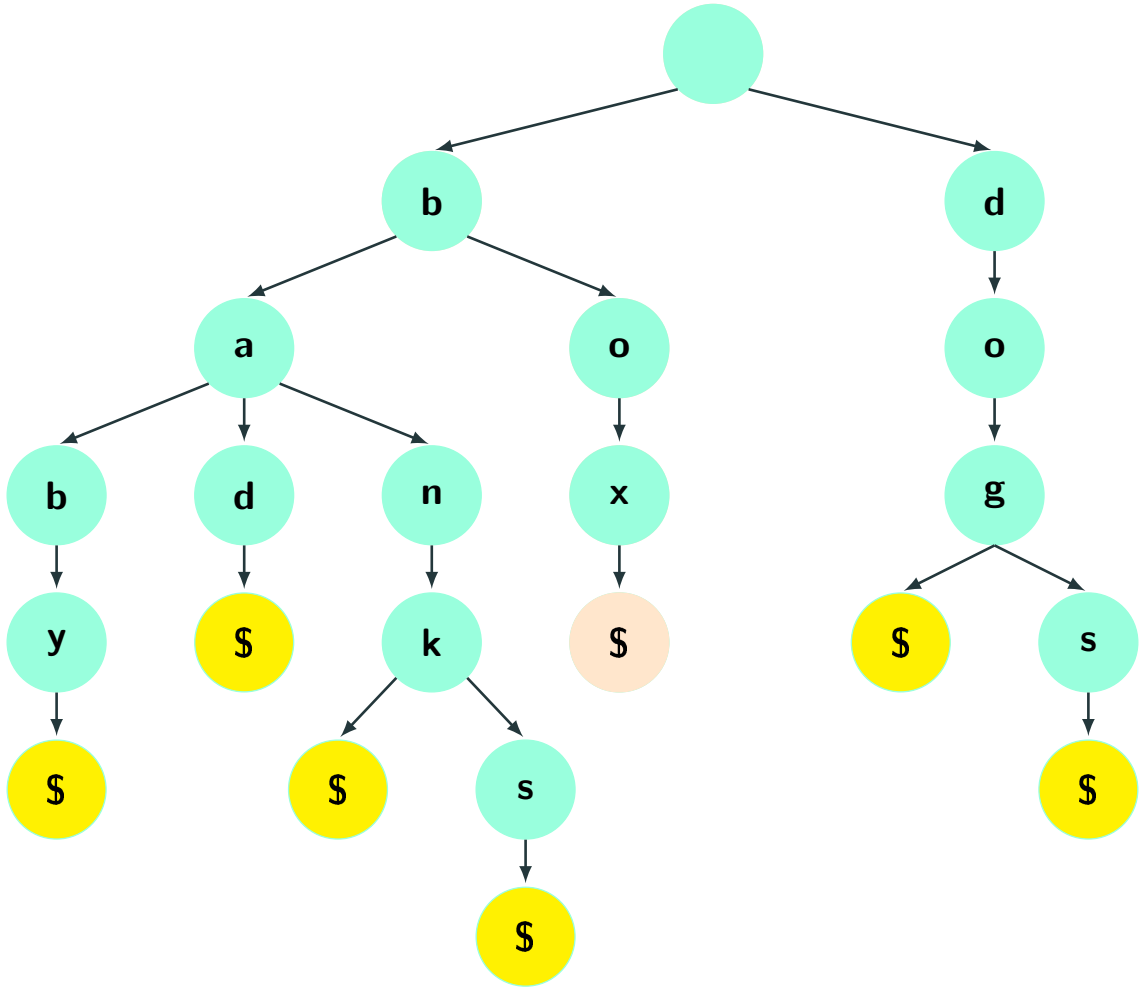
Operations on Trie

Search a String

- Start from the root node
- For each character c in the string (including \$)
  - If a node containing c exists, move to the next node.
    - If $c == \$$, return "Found".
  - Otherwise, return "Not Found".
- Time Complexity: $O(M)$ where M is the length of the string.

Operations on Trie

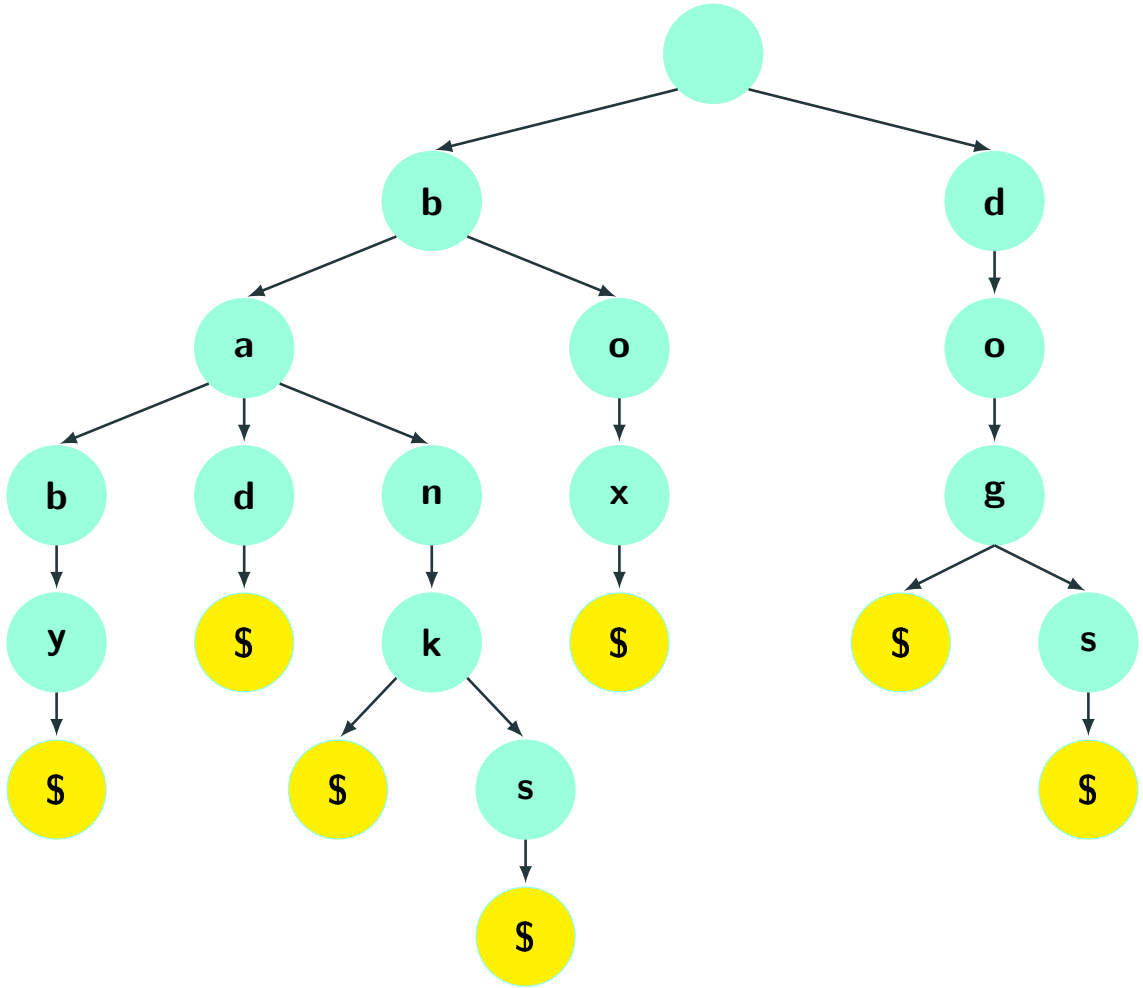
Searching for box:



String box was found.

Operations on Trie

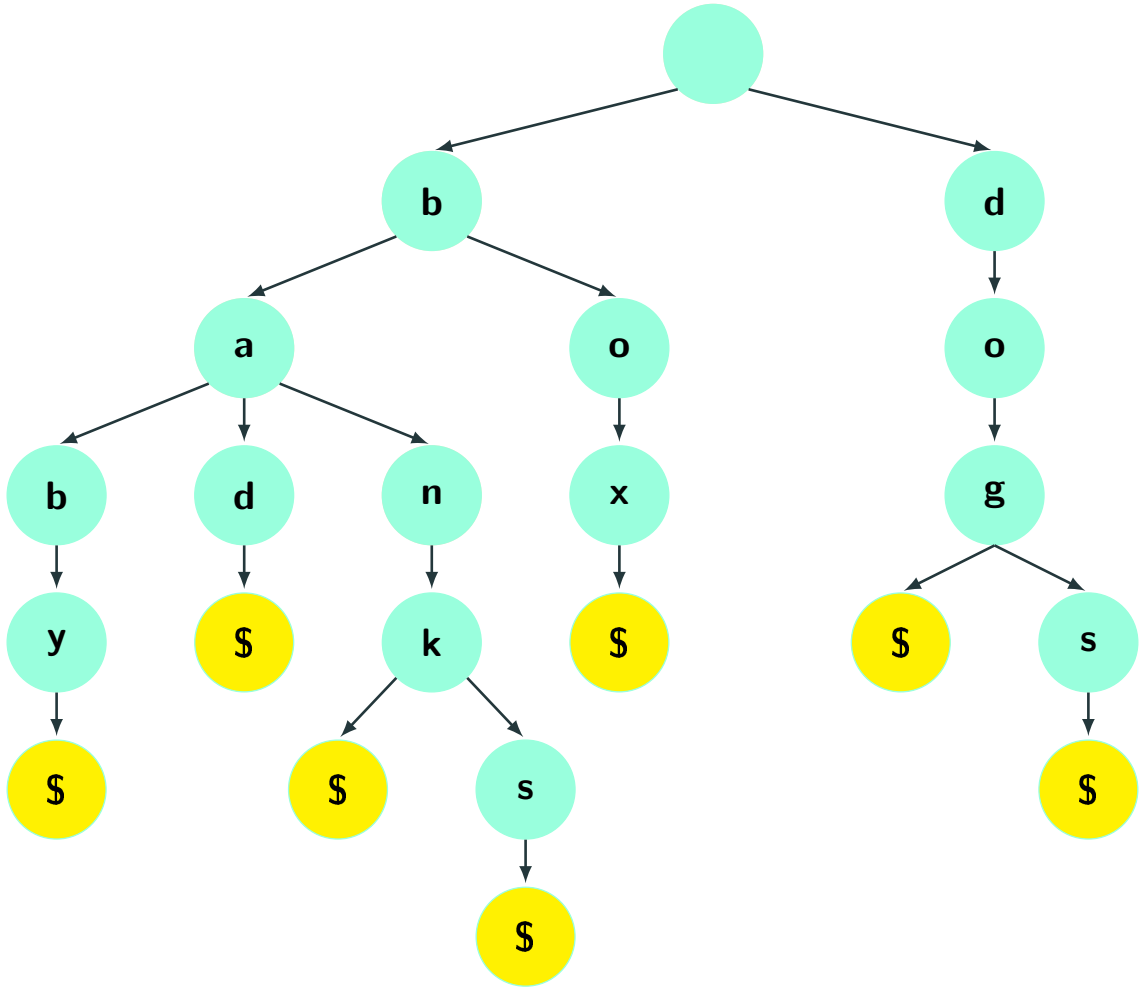
Searching for boxing:



String boxing was not found.

Operations on Trie

Searching for ban:



String **ban** was not found.

Operations on Trie

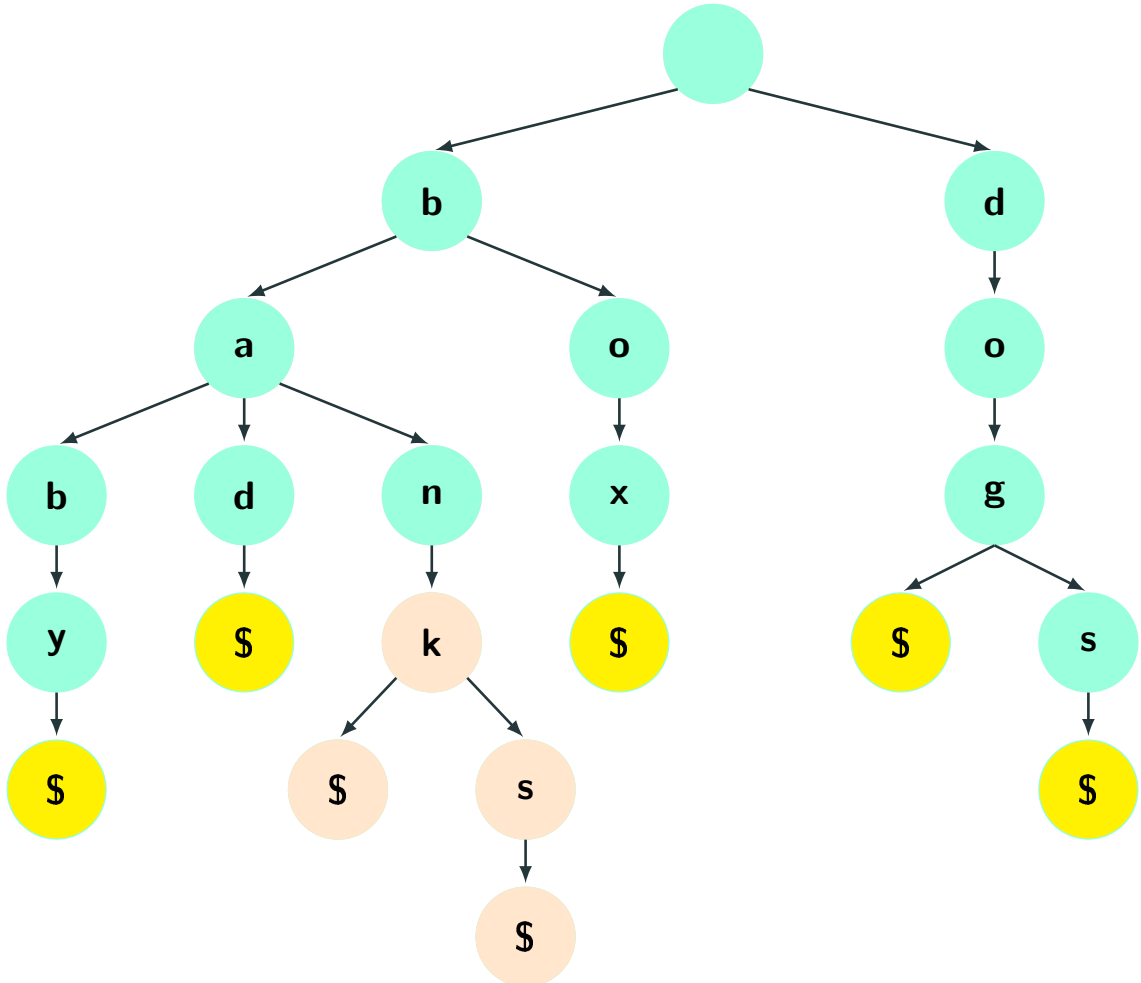
Prefix Matching

- Start from the root node
- For each character c in the prefix
  - If a node containing c exists, move to the node
  - Otherwise, return "Not Found".
- Return all strings in the subtree rooted at the last node

- Time Complexity: $O(M + U)$ where M is the length of the query string and U is the total number of characters in all returned strings.

Operations on Trie

Prefix matching for ban:



Traverse the subtree to get **bank** and **banks**.

Trie Implementation

- At each node, create an array of alphabet size
  - 26 for English letters
  - 4 for DNA strings
- If i^{th} node exists, add pointer at $array[i]$. Otherwise, $array[i] = Nil$.
- The above implementation allows checking whether a node exists or not in $O(1)$ (for constant-sized alphabets).
- Other implementations are possible (e.g., linked lists or hash tables).

Advantages and Disadvantages of Trie

Advantage

- A better search structure than a binary search tree with string keys.
- A more versatile search structure than hash table.
- Prefix matching in $O(M)$ where M is the length of prefix.
- Sorting collection of strings in $O(MN)$ time where M is the length of the longest string and N is the total number of strings.

Disadvantage

- On average tries can be slower (in some cases) than hash tables for looking up patterns/queries.
- Wastes space, since even when a node has few children, you need to create an array of size alphabet.

Some properties of Trie

- The maximum depth is the length of longest string in the collection.
- Insertion, Deletion, Lookup operations take time proportional to the length of the string/pattern being inserted, deleted, or searched.
- But we waste a lot of space if
  - each node has 1 pointer per symbol in the alphabet.
  - deeper nodes typically have mostly null pointers.
- Can reduce total space usage by turning each node into a linked list or binary search tree etc, trading off time for space.

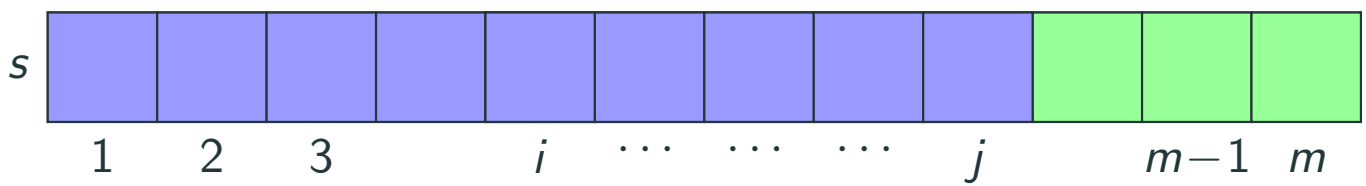
Exercises

- **Longest Prefix Matching:** Given a set of words and a query string s, find the longest prefix of s that matches any word in the Trie.
- **Lexicographic Sorting:** Sort a set of strings in lexicographic order.
- **Autocompletion:** Given a prefix string s, list the top k most frequent completions, i.e. strings that are have prefix s.
- **Text Search:** Search for a string s within a large text or document.

Suffix Trie

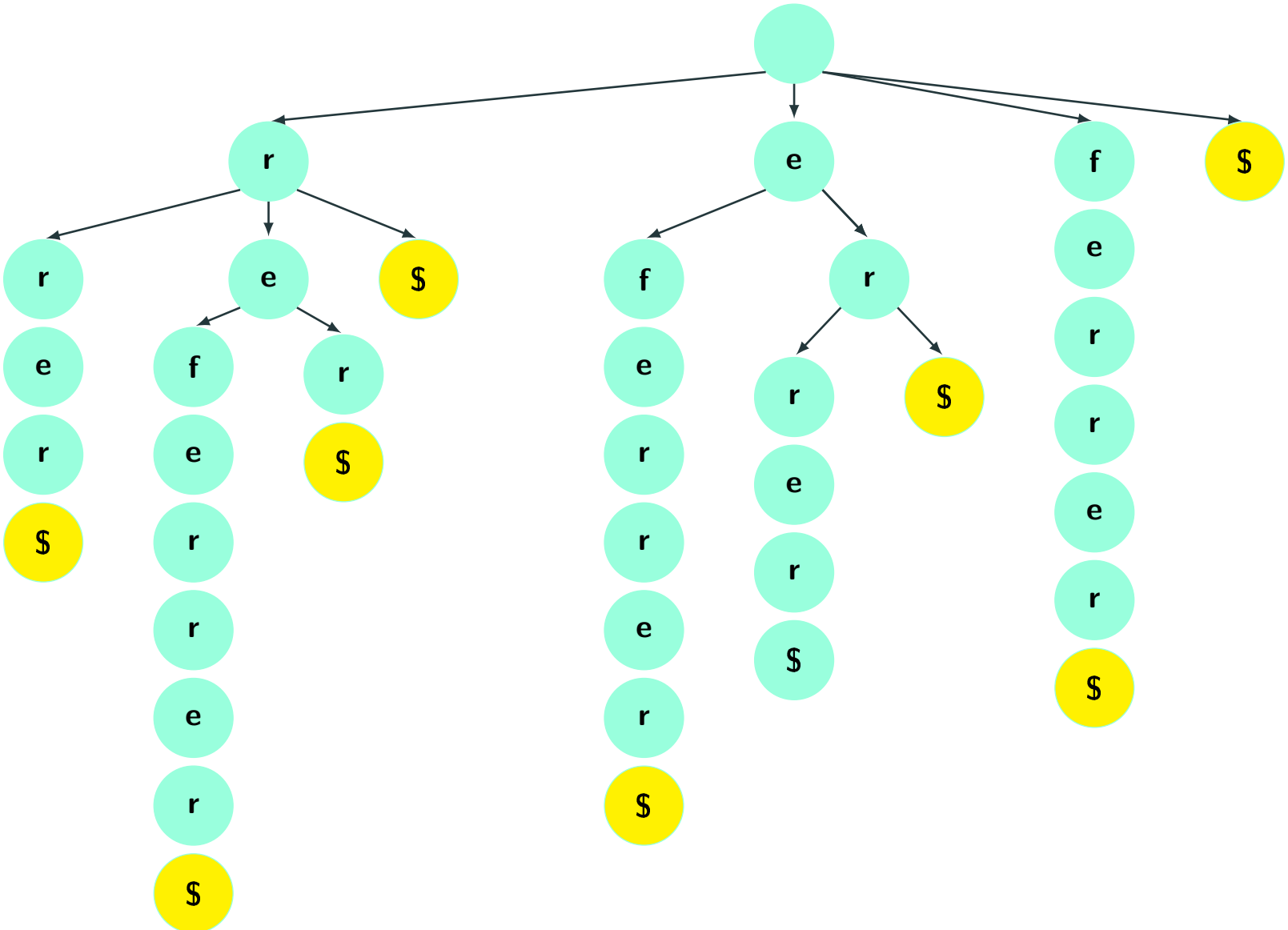
Prefixes and Suffixes

- Goal: make a prefix trie of suffixes to efficiently search substrings.
- (Prefix) Tries are very useful for quickly looking up whole words, but more generally, **prefixes** of words
- A prefix of a word $s[1..m]$ is some string $s[1..i]$ where $1 \leq i \leq m$
- A suffix of a word $s[1..m]$ is $s[i..m]$ where $1 \leq i \leq m$.
- Any substring of a word is a prefix of some suffix.



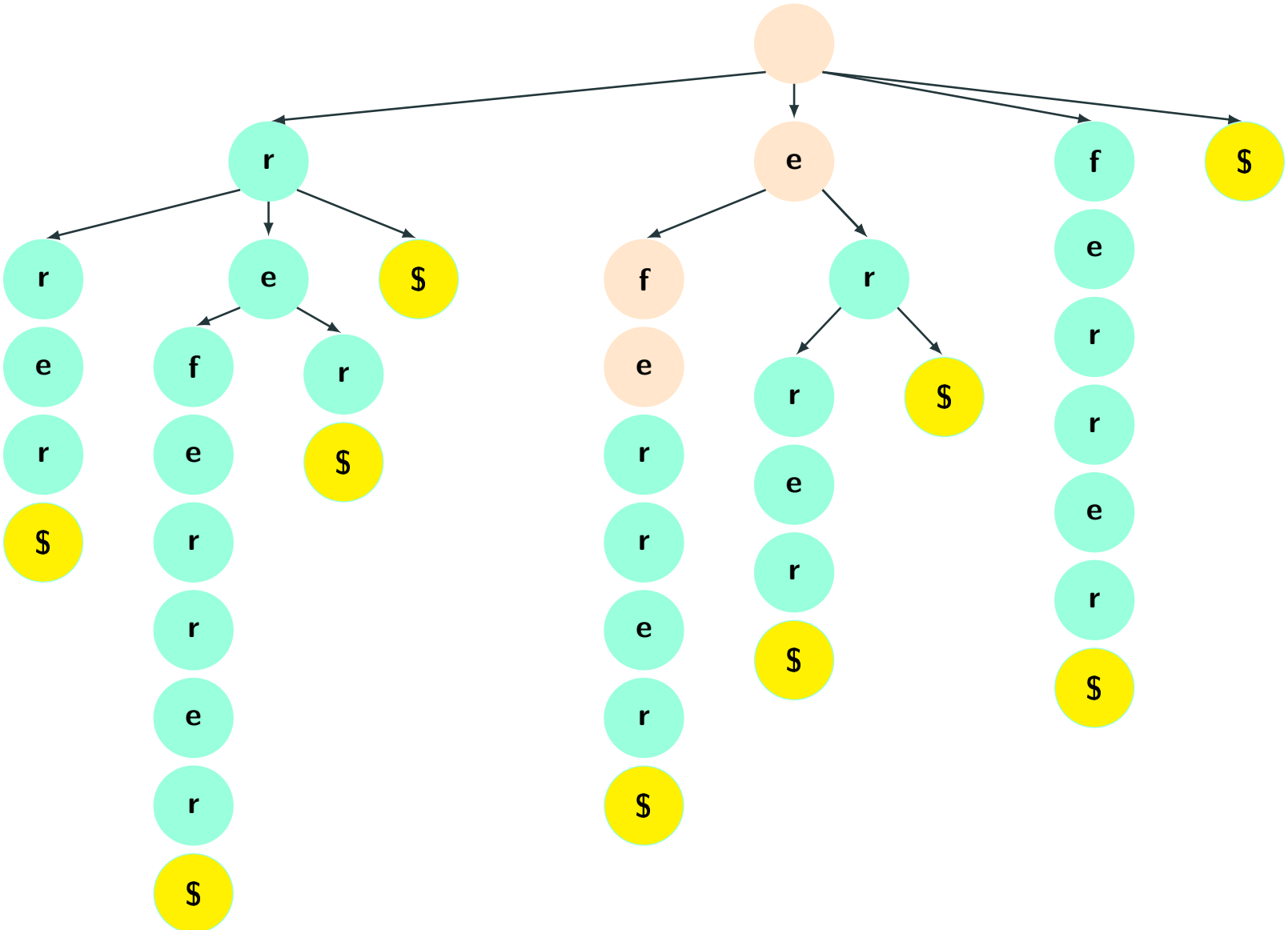
Suffix Tries

A trie constructed using all suffixes of the text is called a Suffix Trie.



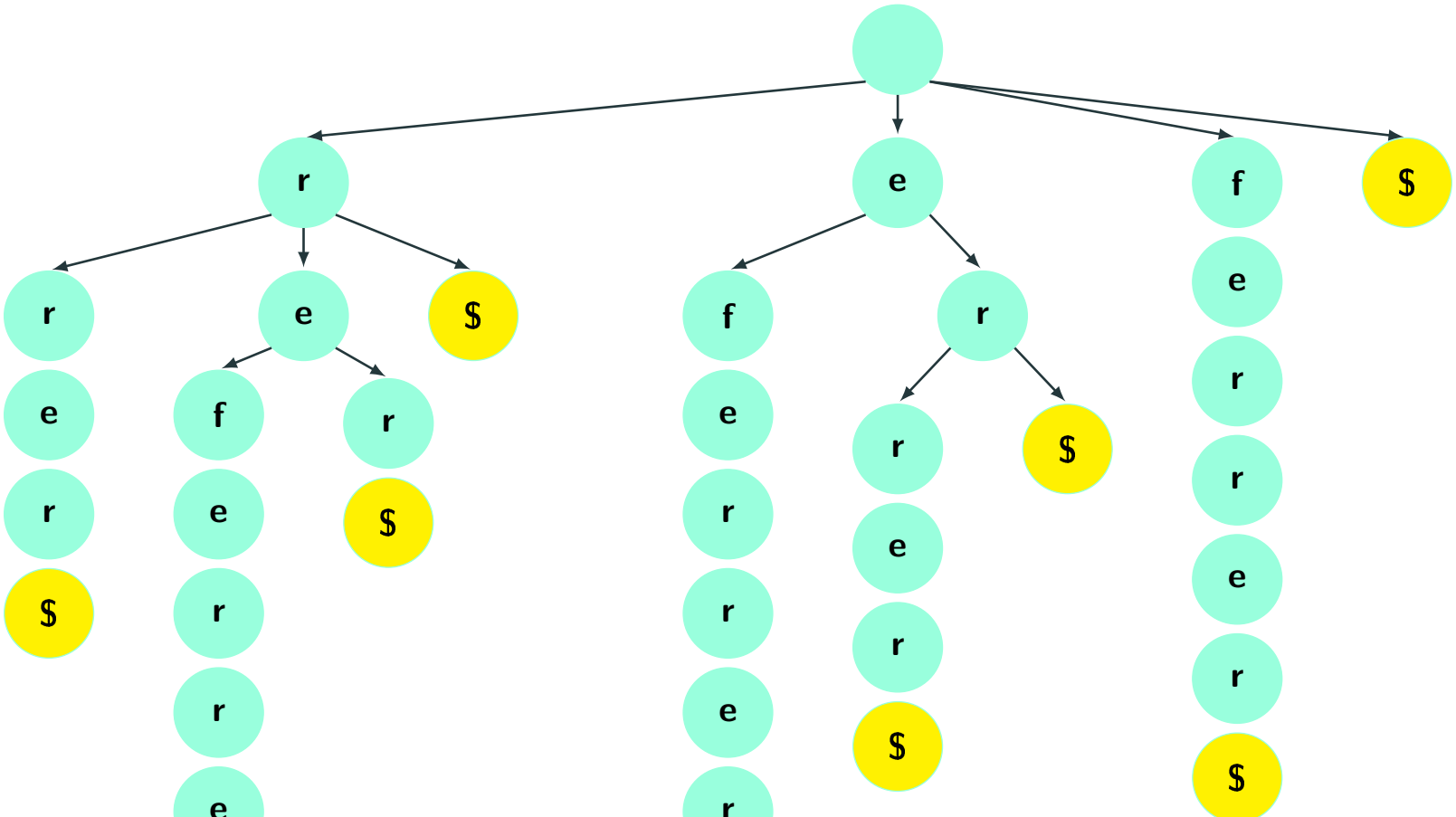
Suffix Tries

A substring “efe” of “referrer” traces a path from root to some node.



Construct Suffix Tries

Insert all suffix of “referrer” in the trie. Insert suffix “referrer$” into the trie. Insert suffix “eferrer$” into the trie. Insert suffix “ferrer$” into the trie. Insert suffix “errer$” into the trie. Insert suffix “rrer$” into the trie. Insert suffix “rer$” into the trie. Insert suffix “er$” into the trie. Insert suffix “r$” into the trie. Insert suffix “$” into the trie.



Operations on Suffix Trie

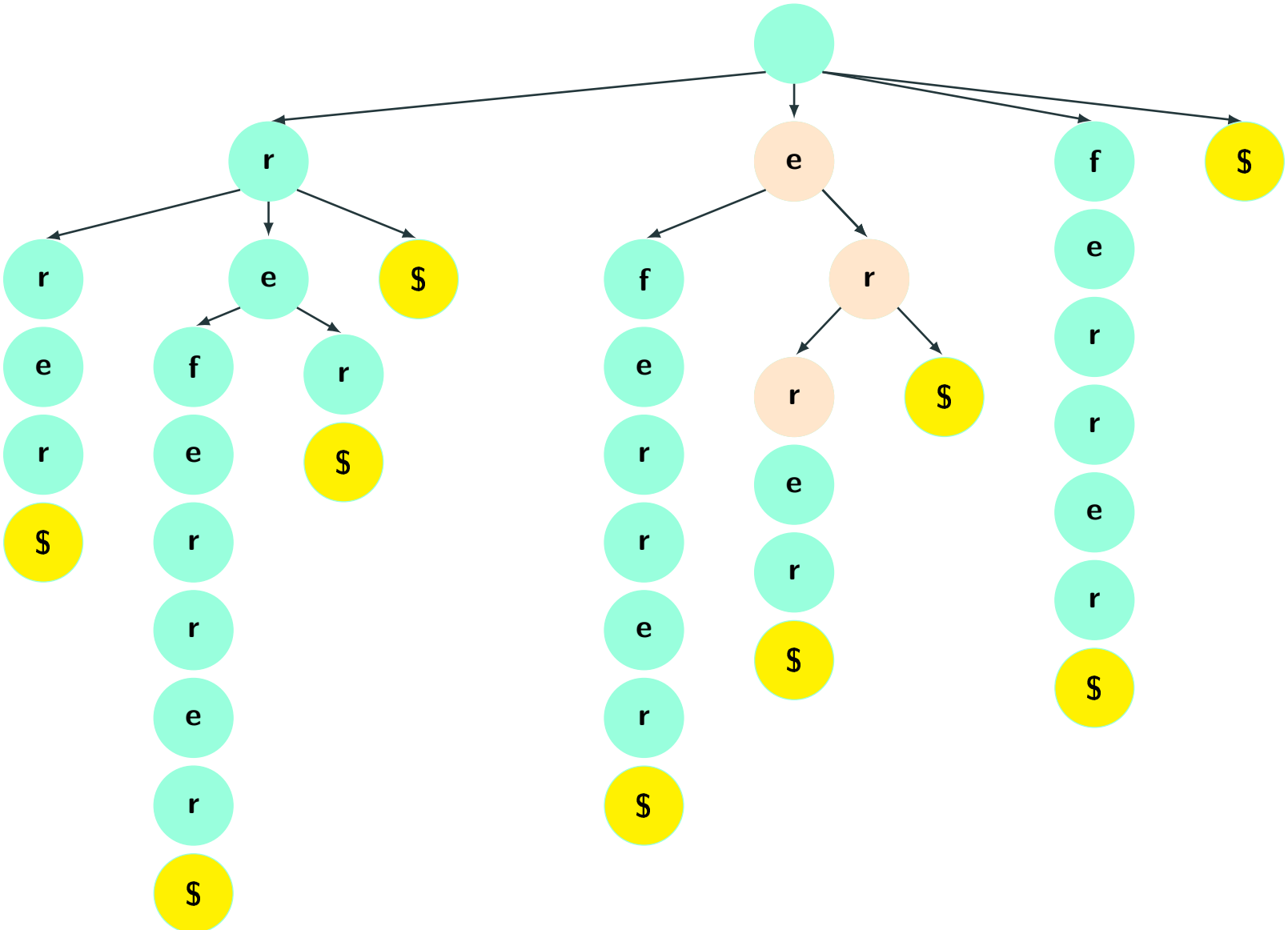
Substring Search

- Start from the root node
- For each character c in the prefix
  - If a node containing c exists, move to the node
  - Otherwise, return "Not Found".
- Return "Found".

- Time Complexity: $O(M)$ where M is the length of the query string.

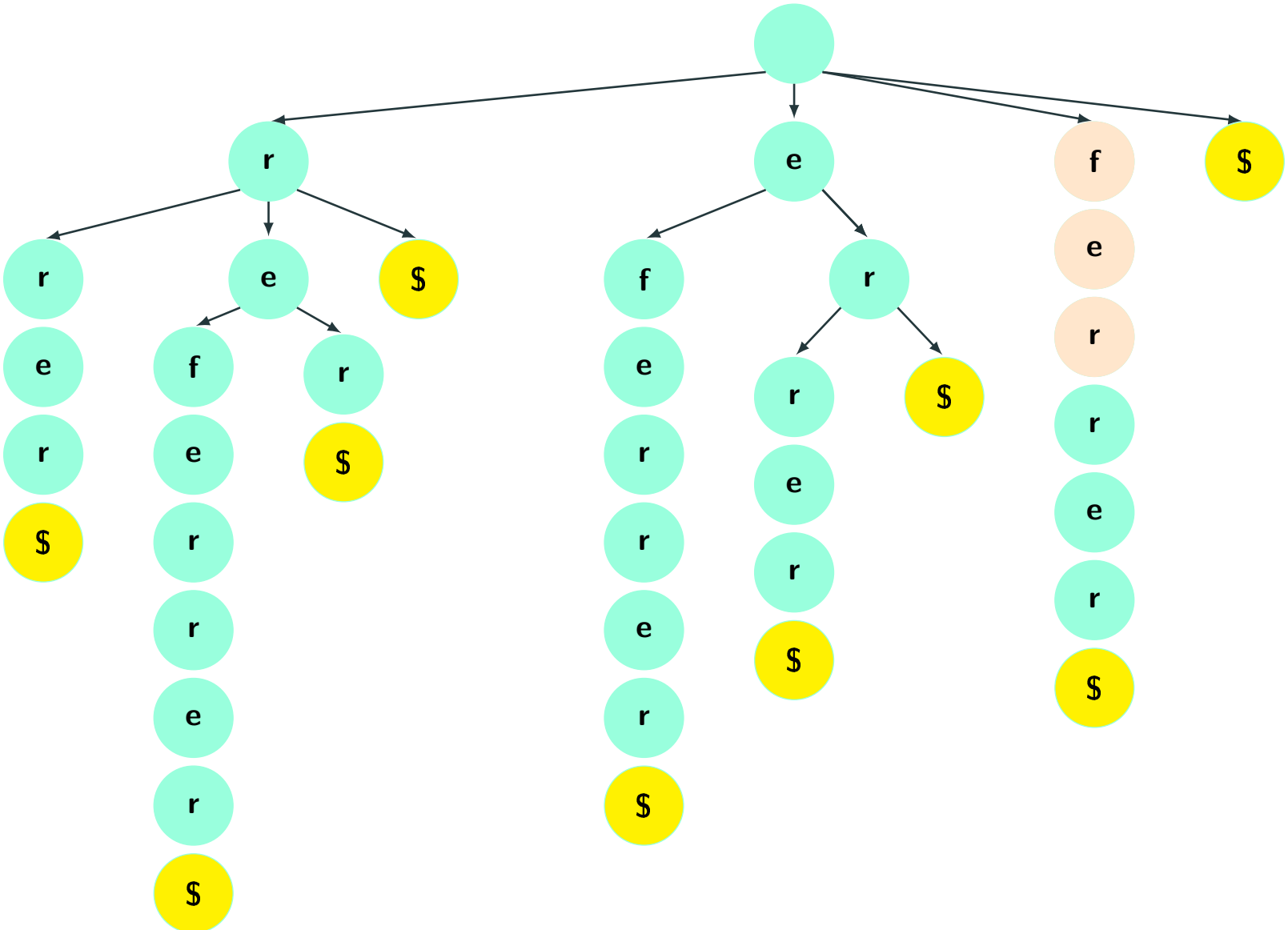
Operations on Suffix Trie

Substring search: find “err” in the suffix tree. Found!



Operations on Suffix Trie

Substring search: find "fers" in the suffix tree. Not found.



Operations on Suffix Trie

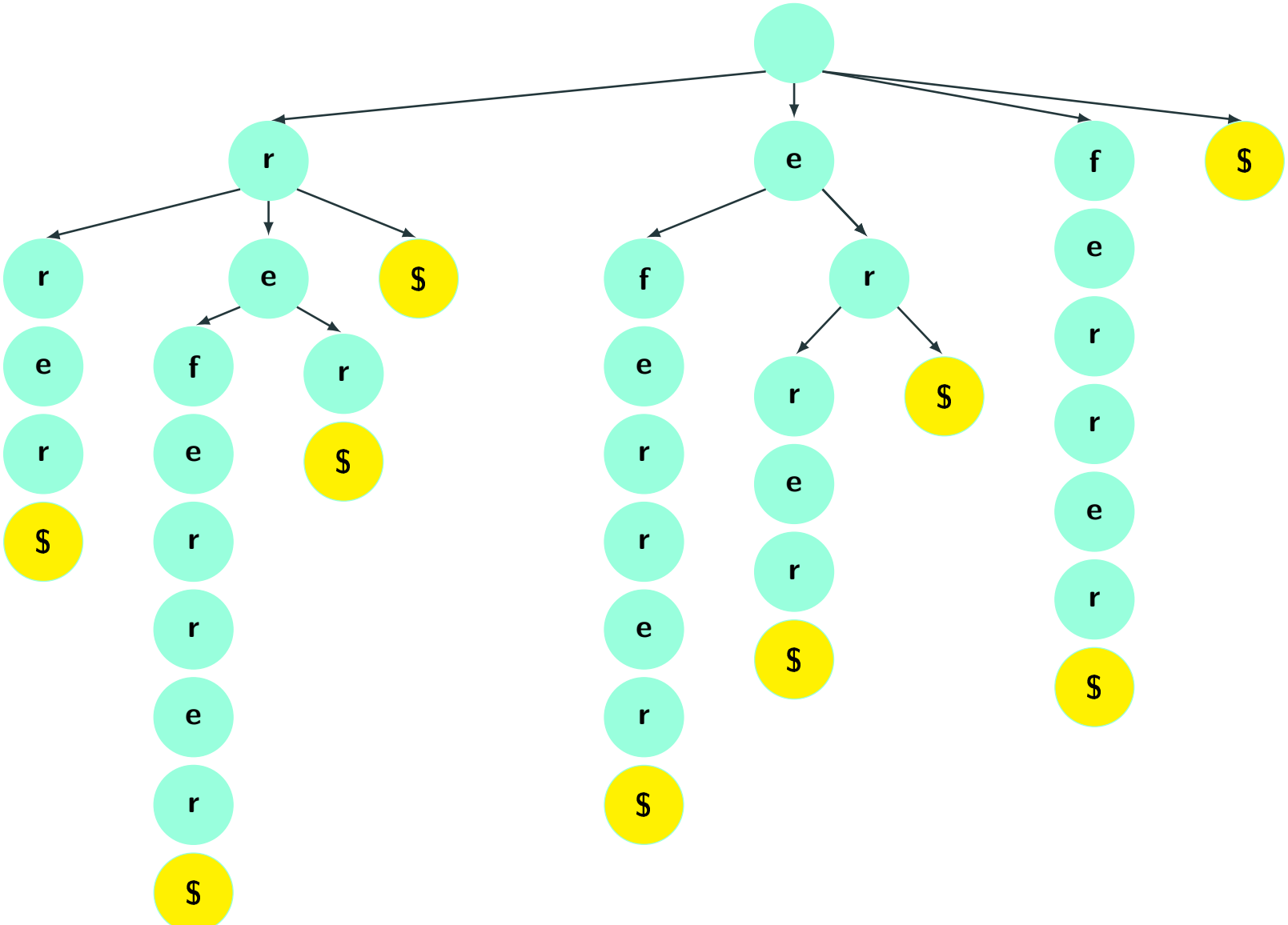
Counting Occurrences

- Start from the root node
- For each character c in the prefix
  - If a node containing c exists, move to the node
  - Otherwise, return "Not Found".
- Traverse the subtree and count the number of leaf nodes (\$).

- Time Complexity: ?

Operations on Suffix Trie

Count occurrences of “er”



Operations on Suffix Trie

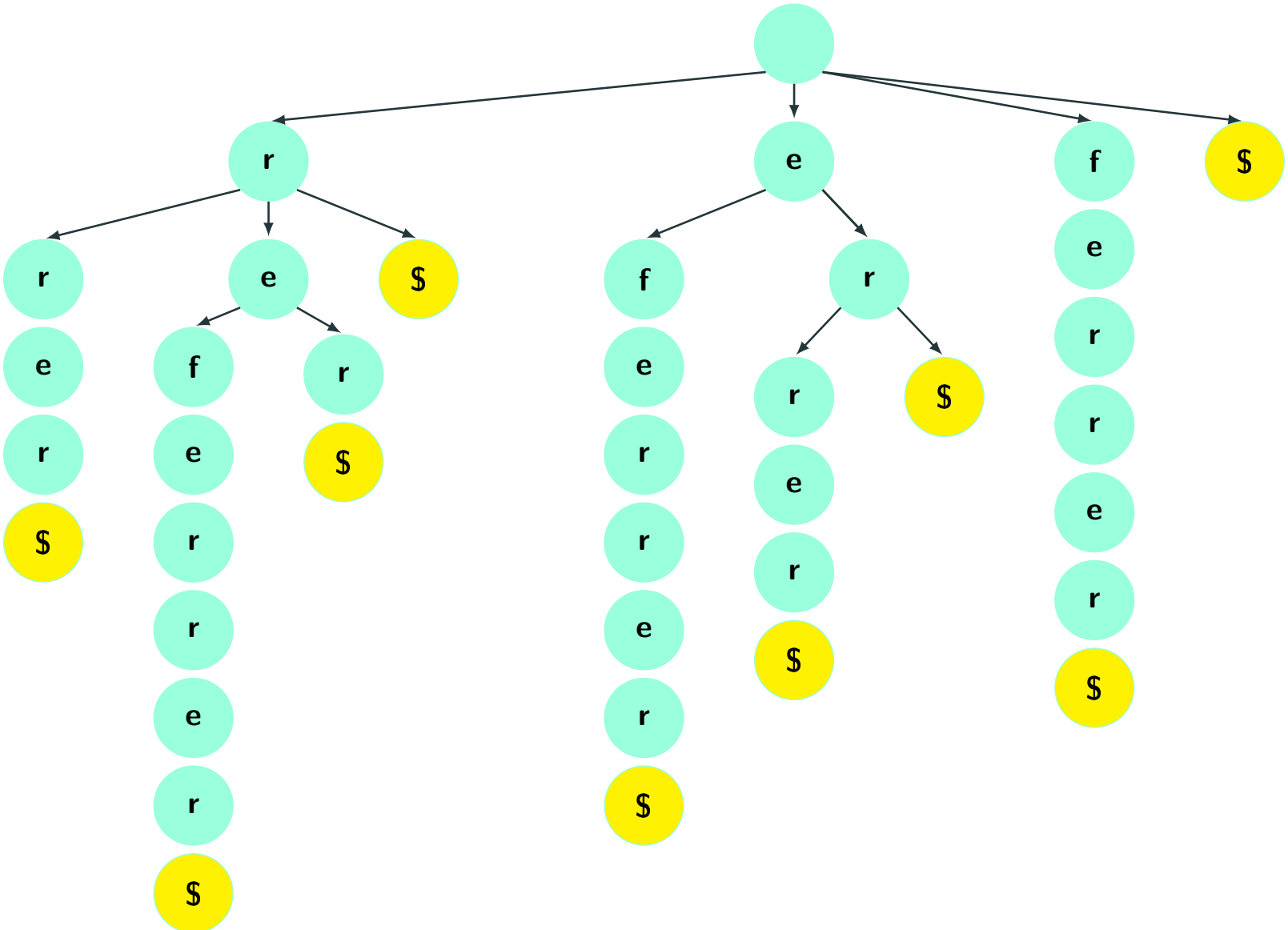
Counting Occurrences

- Start from the root node
- For each character c in the prefix
  - If a node containing c exists, move to the node
  - Otherwise, return "Not Found".
- Traverse the subtree and count the number of leaf nodes (\$).

- Time Complexity: $O(M)$ where M is the length of the query string if the number of leaf nodes is maintained during insertion of strings.

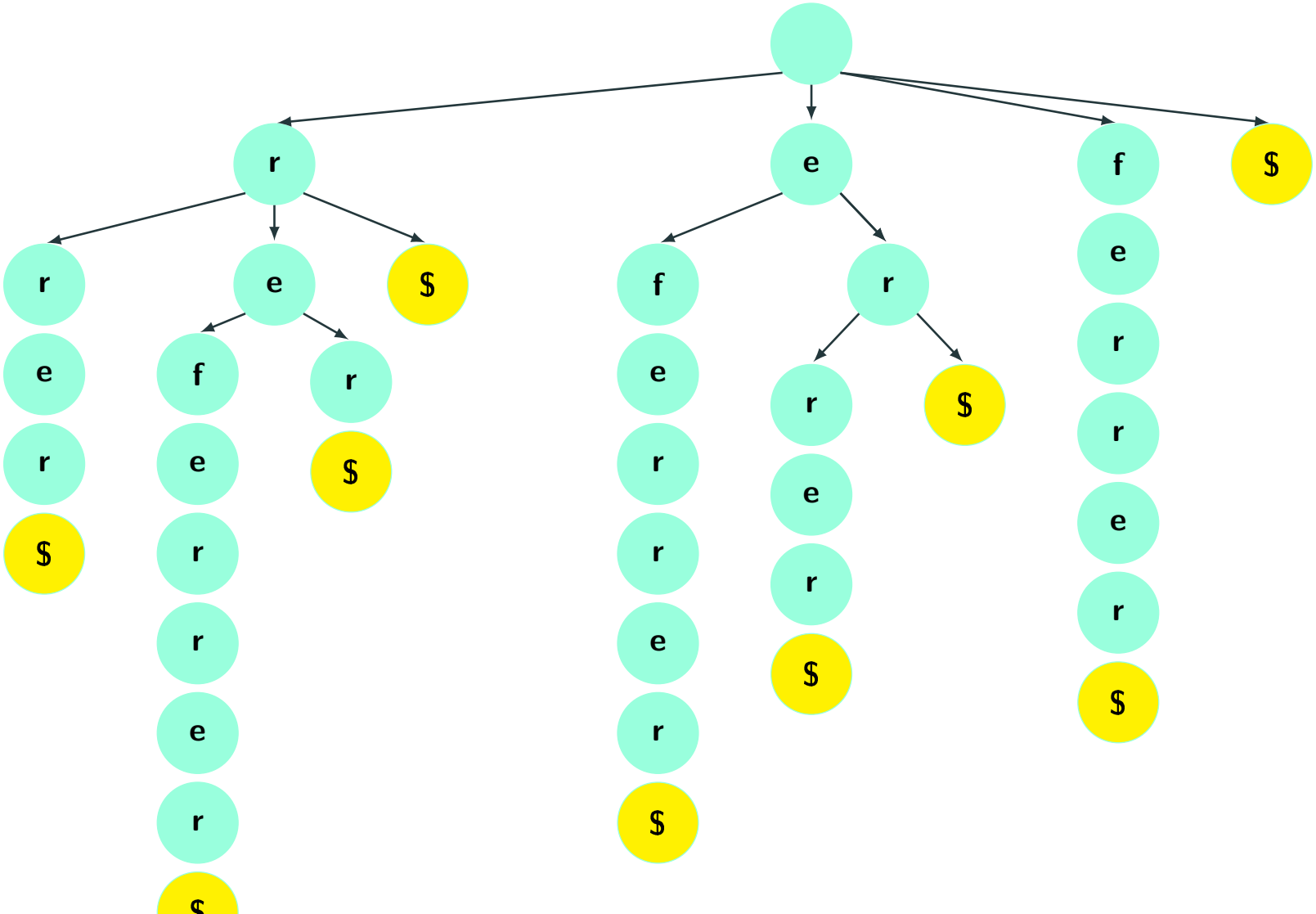
Operations on Suffix Trie

Find the longest repeated substring?



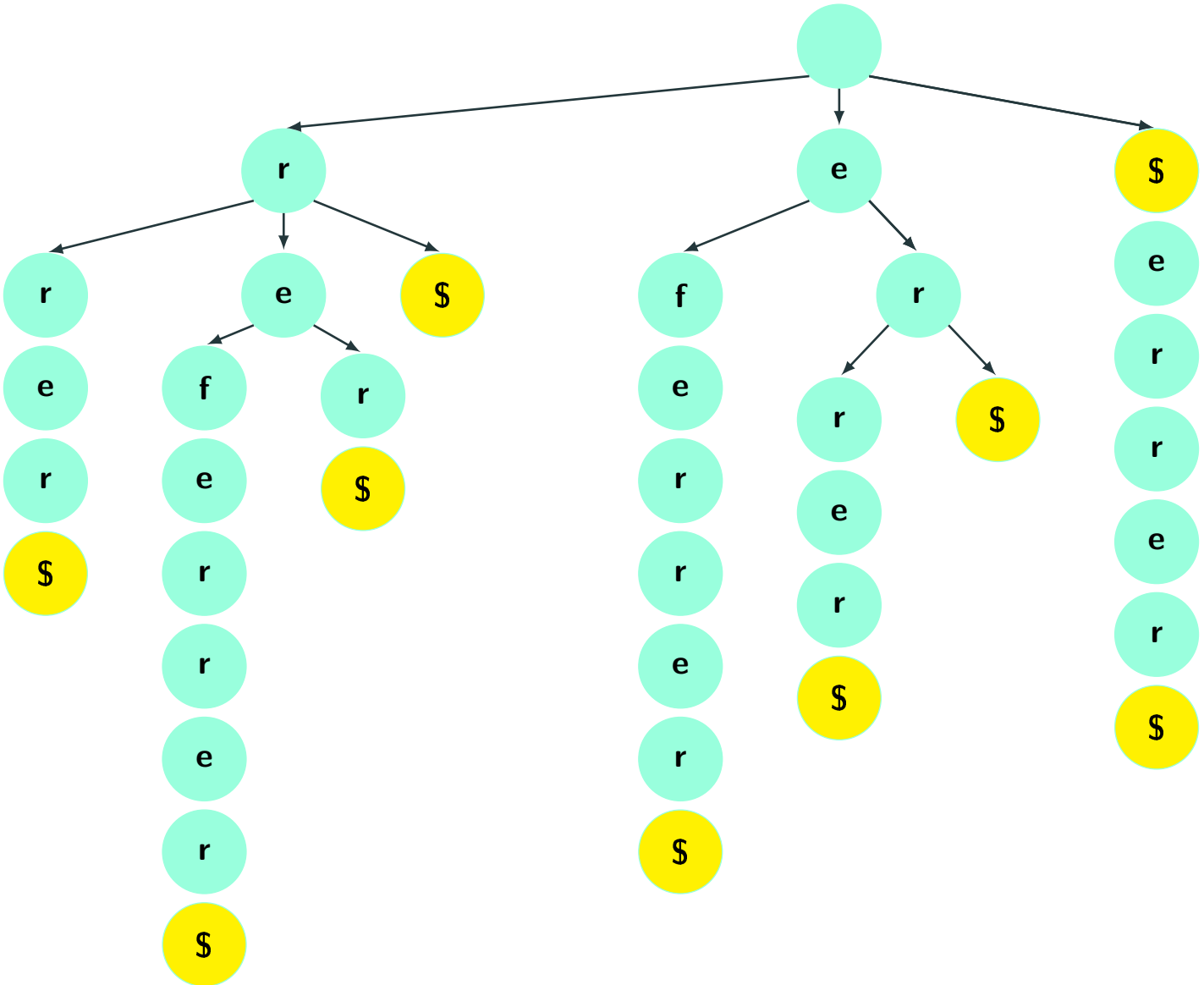
Space Complexity of Suffix Trie

Let N be the size of the string for which suffix trie is constructed, what is the space complexity?total space cost: $O(N^2)$



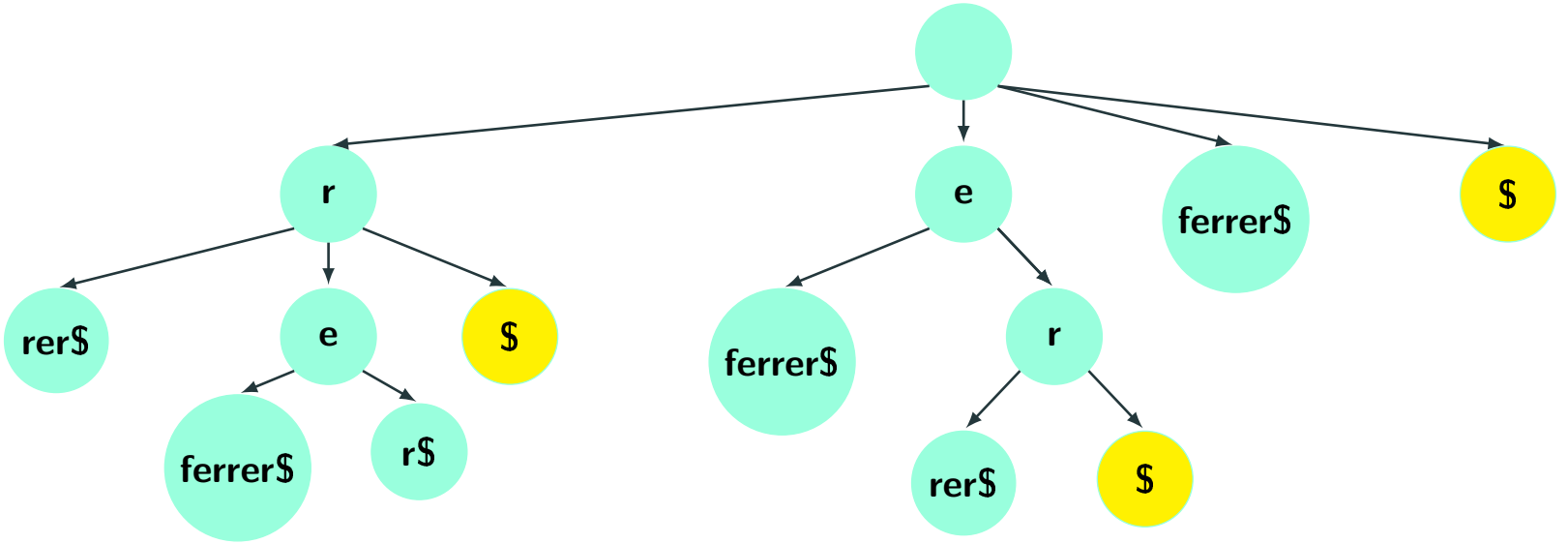
Suffix Tree

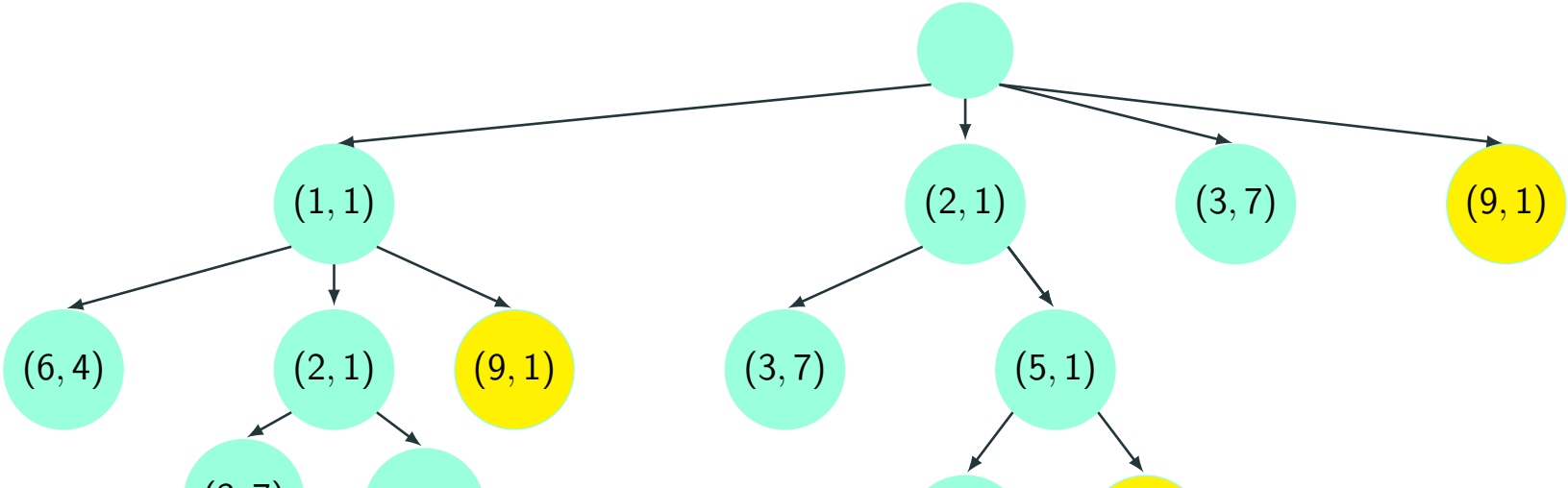
Compress branches by merging the nodes that have only one child.



Suffix Tree

Replace every substring with numbers (x, y) where x is the starting index of the substring and y is its length.





Space Complexity of Suffix Tree

- There are exactly $N + 1$ leaves.
- Every (internal) node has at least 2 children. (Why?)
- There are at most N internal nodes.
- Each node stores two integers, so the space is bounded by $O(N)$.

Time Complexity of Constructing Suffix Tree

- The algorithm described earlier inserts $O(N)$ suffixes
- Insertion cost of each suffix is linear in the size of suffix
- Average suffix size is $O(N)$
- Compressing the trie requires traversing it, $O(N^2)$
- Thus, total time complexity to construct tree is $O(N^2)$
- Ukkonen's algorithm is able to construct a suffix tree in $O(N)$!

Exercises

- **Text Search:** Find all occurrences of a pattern string s within a large text or document.
- **Longest Repeated Substring:** Given a string $S[1..n]$, find the longest substring that appears at least twice in S.
- **Longest Common Substring (Two Strings):** Given two strings s_1 and s_2, find the longest substring common to both.
- **Longest Common Substring (Multiple Strings):** Given a set of strings $S = \{s_1, s_2, \ldots, s_n\}$, find the longest substring that occurs in at least k of them.

Reading

- Course Notes: Chapter 12 and 13
- You can also check algorithms' textbooks for contents related to this lecture, e.g.:
  - CLRS: Chapters 18
- For a more advanced treatment of trie and suffix trees: Dan Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge University Press.

Concluding remarks

- Take home message
  - Tries and suffix trees provide efficient text search and pattern matching (typically linear in number of characters in string)
  - Linear time construction for both is possible (beyond the scope of this unit)
- Things to do:
  - Implement tries and suffix trees
  - Run various queries on the data structures
- Coming up next:
  - AVL Trees, 2-3 Search Trees and Red Black Trees